

name'. In the above script, we probed the `'execve'` system call, which is used to start a new process, and `'exit'`, which is used to exit from a process. Instead of a simple `print` statement, we have used arguments with `printf`. The function `execname()` returns the name of the current process, `pid()` the process ID, and `argvstr()` the command-line arguments list. Run the script with `stap`, as follows:

```
# stap exec.stp
```

After this, I started `gnome-system-monitor` and closed it, causing the following output:

```
gnome-panel(12515) exit (0)
gnome-panel(12516) execve (/usr/lib/qt-3.3/bin/gnome-system-monitor )
gnome-panel(12516) execve (/usr/local/bin/gnome-system-monitor )
gnome-panel(12516) execve (/usr/bin/gnome-system-monitor )
gnome-system-no(12516) exit (0)
```

Probing a kernel function

The most important thing SystemTap can do is probe a kernel function by name. From `syscall` probing, we get only limited details; there are thousands of other kernel functions too, that we may need to probe—like network stack functions. The syntax for a kernel function probe is given below:

```
probe kernel.function("function-name") {}
```

I have chosen to probe `'ip_rcv()'` as it is called very frequently, whenever an IP packet is received:

```
# cat ip_rcv.stp
probe kernel.function("ip_rcv")
{
    printf ("packet rcvd %s\n", $parms);
}
probe timer.ms(4000)
{
    exit()
}
```

In the kernel function probe handler, we have used the argument `'$parms'` which returns the function's parameters and its values. Other options are `'$locals'` (local variables) and `'$vars'` (all variables). The second handler in this script is a timer probe, whose syntax is as follows:

```
probe timer.ms(milliseconds)
```

Our handler will run after 4000 milliseconds—thus it will `exit()` after 4 seconds. Now run the following script:

```
# stap ip_rcv.stp
packet rcvd skb=0xf50e9f00 dev=0xf412d000 pt=0xc0a4e760 orig_
```

```
dev=0xf412d000
packet rcvd skb=0xf272ae00 dev=0xf412d000 pt=0xc0a4e760 orig_
dev=0xf412d000
packet rcvd skb=0xf6cefab0 dev=0xf412d000 pt=0xc0a4e760 orig_
dev=0xf412d000
packet rcvd skb=0xf436a240 dev=0xf412d000 pt=0xc0a4e760 orig_
dev=0xf412d000
.....
```

Using jiffies

If you are a kernel programmer, you will be familiar with `jiffies`, a global kernel variable representing the number of ticks since the machine has booted. This can be used instead of milliseconds, in the timer. The syntax is given below:

```
probe timer.jiffies(jiffies){}
```

Here's a test script and its execution:

```
# cat variable.stp
global counter
probe timer.jiffies(100) {
    printf("count = %d\n", counter++);
}
# stap variable.stp
count = 0
count = 1
count = 2
count = 3
```

The handler will be called every 100 ticks. Like in C, we can declare global variables in the script—as we have declared `'count'` and used it in the handler.

Probing kernel functions in a C file

At some time we may need to probe all functions present in a kernel C file. For example, we may want to know which functions are called during the IP input layer. The syntax is `probe kernel.function(function-name@filename)` and the test script is as follows:

```
# cat ip_input.stp
probe kernel.function("@net/ipv4/ip_input.c")
{
    printf ("ip_input-> time=%u function = %s\n", gettimeofday_s(),
probefunc());
}
probe timer.ms(10000)
{
    exit()
}
```

Here we have used the file `net/ipv4/ip_input.c` and `'*'` instead of a function name, to invoke the handler when any function in this file is called. Now, run the following script: